Bharatiya Vidya Bhavan's

SARDAR PATEL INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering — Cryptography and Network Security Laboratory

Experiment: Secure E-mail using PGP and S/MIME

Field	Value
Name	Anusha Dalal
UID	2024300032
Date of experiment	31 Oct 2026

Aim

To implement secure message exchange using Pretty Good Privacy (PGP/OpenPGP) and S/MIME, demonstrating public-key management, digital signatures, confidentiality through encryption, signature verification, decryption, and detection of message tampering.

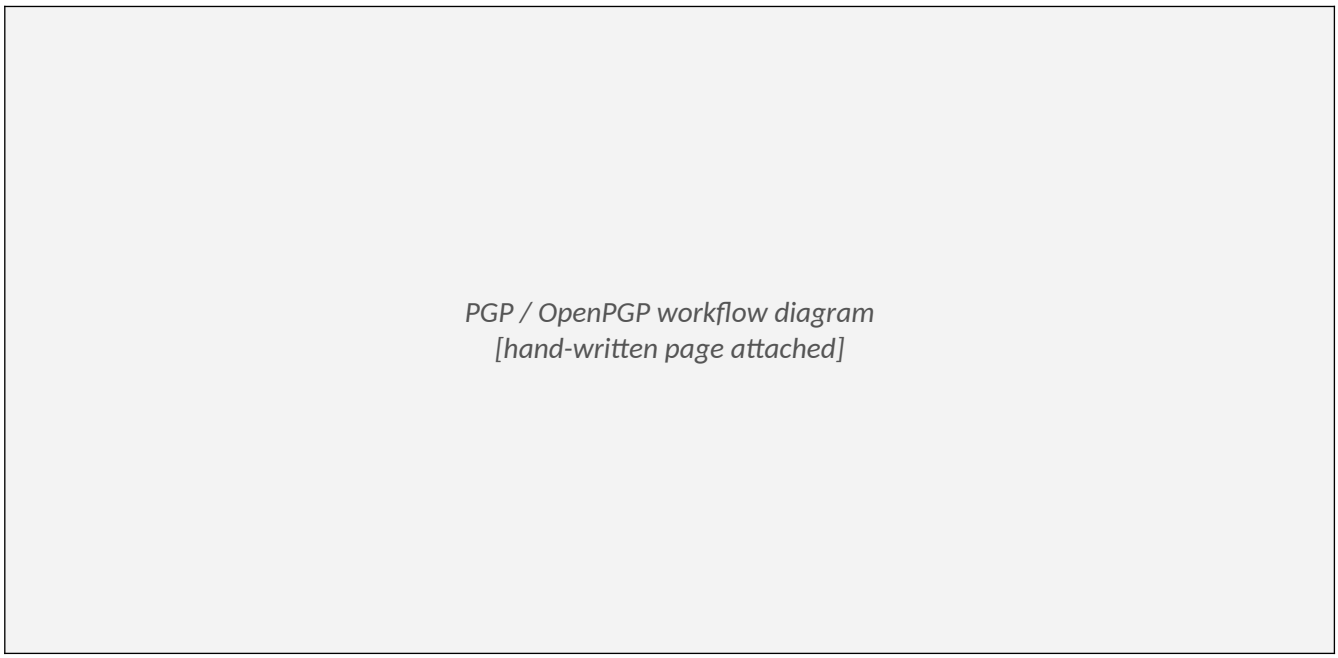
Contents (order of Section 8)

#	Item
1	Hand-drawn PGP and S/MIME workflow diagrams
2	Table 1: execution environment
3	PGP evidence: key listings and fingerprints, export/import, detached signature, encryption/decryption, signed-and-encrypted message, tampering
4	S/MIME evidence: CA and user certificates, signed message, verification, encrypted message, decryption, tampering and untrusted-certificate results
5	Tables 2 and 3, comparative analysis, answers to the discussion questions, conclusion
6	Commands used and deviations caused by software versions

Every terminal capture shows the name and UID line and the prompt of the machine used (anush@anusha). No private keys are included in this report.

1. Hand-drawn workflow diagrams (Part A)

Drawn on paper before using the computer: Sender (Alice), Receiver (Bob), the insecure channel, key pairs/certificates, signing, encryption, decryption, verification, the ACCEPT/REJECT decision, an attacker modifying the message, and private keys never transmitted.





2. Table 1: Execution environment

Parameter	Observed value
Operating system and version	Ubuntu 24.04.5 LTS (kernel 6.6.87.2-microsoft-standard-WSL2)
GnuPG version	GnuPG 2.4.4 (libgcrypt 1.10.3)
OpenSSL version	OpenSSL 3.0.13 30 Jan 2024
CPU / RAM (optional)	12th Gen Intel(R) Core(TM) i5-1235U, 12 logical CPUs / 3.7 GiB
Date of experiment	Sat Oct 31 16:35:33 UTC 2026

Figure 1: Table 1 evidence: operating system, kernel, GnuPG and OpenSSL versions, CPU, RAM, date



3. PGP / OpenPGP evidence (Part B)

Alice and Bob each have a separate GnuPG home directory, so each private key stays in its own keyring. Only public keys, the message, the signature and encrypted files are placed in the transfer folder.

Figure 2: Step B1: GnuPG installed - gpg --version



Figure 3: Step B2: separate working directories and GnuPG homes for Alice and Bob

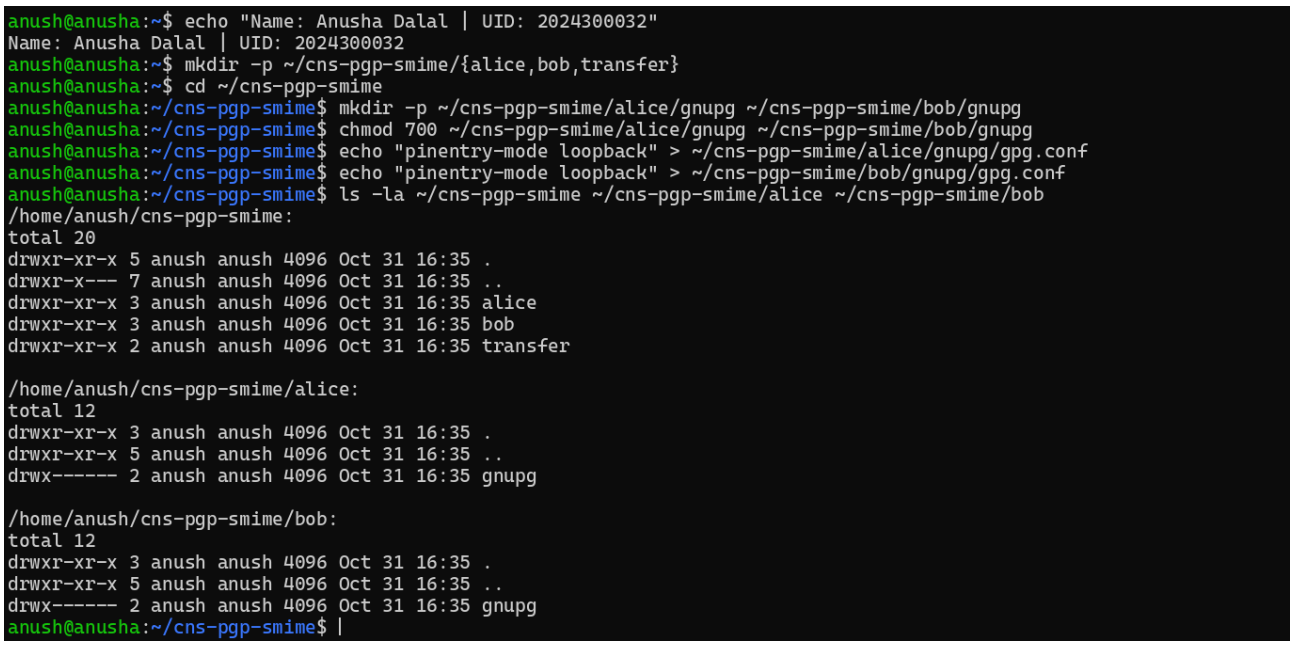


Figure 4: Step B3: Alice's key pair (RSA and RSA, 3072 bits) - gpg --full-generate-key, list-keys, list-secret-keys (1/2)

```
anush@anusha:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg
anush@anusha:~/cns-pgp-smime$ gpg --full-generate-key
gpg (GnuPG) 2.4.4; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: keybox '/home/anush/cns-pgp-smime/alice/gnupg/pubring.kbx' created
Please select what kind of key you want:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 3072
Requested keysize is 3072 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 1y
Key expires at Sun Oct 31 16:35:33 2027 UTC
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Alice CNS Lab
Email address: alice-lab@example.test
Comment:
You selected this USER-ID:
    "Alice CNS Lab <alice-lab@example.test>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? O
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
Enter passphrase:                  We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/anush/cns-pgp-smime/alice/gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/anush/cns-pgp-smime/alice/gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/anush/cns-pgp-smime/alice/gnupg/openpgp-revocs.d/2EB7F49C23F32EA04E65C5
B841C1BACD4CEEF430.rev'
public and secret key created and signed.

pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      2EB7F49C23F32EA04E65C5B841C1BACD4CEEF430
uid                      Alice CNS Lab <alice-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
anush@anusha:~/cns-pgp-smime$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
```

Figure 5: Step B3: Alice's key pair (RSA and RSA, 3072 bits) - gpg --full-generate-key, list-keys, list-secret-keys (2/2)

```
gpg: next trustdb check due at 2027-10-31
/home/anush/cns-pgp-smime/alice/gnupg/pubring.kbx
-------------------------------------------------
pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      2EB7F49C23F32EA04E65C5B841C1BACD4CEEF430
uid           [ultimate] Alice CNS Lab <alice-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
anush@anusha:~/cns-pgp-smime$ gpg --list-secret-keys
/home/anush/cns-pgp-smime/alice/gnupg/pubring.kbx
-------------------------------------------------
sec   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      2EB7F49C23F32EA04E65C5B841C1BACD4CEEF430
uid           [ultimate] Alice CNS Lab <alice-lab@example.test>
ssb   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
anush@anusha:~/cns-pgp-smime$
```

Figure 6: Step B3: Bob's key pair (RSA and RSA, 3072 bits) and key listing (1/2)

```
anush@anusha:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg
anush@anusha:~/cns-pgp-smime$ gpg --full-generate-key
gpg (GnuPG) 2.4.4; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: keybox '/home/anush/cns-pgp-smime/bob/gnupg/pubring.kbx' created
Please select what kind of key you want:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 3072
Requested keysize is 3072 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 1y
Key expires at Sun Oct 31 16:35:37 2027 UTC
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Bob CNS Lab
Email address: bob-lab@example.test
Comment:
You selected this USER-ID:
    "Bob CNS Lab <bob-lab@example.test>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? O
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
Enter passphrase:                   We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.

gpg: /home/anush/cns-pgp-smime/bob/gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/anush/cns-pgp-smime/bob/gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/anush/cns-pgp-smime/bob/gnupg/openpgp-revocs.d/61B4DEFA64CB87B479DE9CB7
04376E3027539C54.rev'
public and secret key created and signed.

pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      61B4DEFA64CB87B479DE9CB704376E3027539C54
uid                      Bob CNS Lab <bob-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
anush@anusha:~/cns-pgp-smime$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
```

Figure 7: Step B3: Bob's key pair (RSA and RSA, 3072 bits) and key listing (2/2)

```
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
gpg: next trustdb check due at 2027-10-31
/home/anush/cns-pgp-smime/bob/gnupg/pubring.kbx
------------------------------------------------
pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      61B4DEFA64CB87B479DE9CB704376E3027539C54
uid           [ultimate] Bob CNS Lab <bob-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
anush@anusha:~/cns-pgp-smime$
```

Figure 8: Step B4: export public keys only, import into the other keyring, compare fingerprints

```
anush@anusha:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg
anush@anusha:~/cns-pgp-smime$ gpg --armor --export alice-lab@example.test > ~/cns-pgp-smime/alice_public.asc
anush@anusha:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg
anush@anusha:~/cns-pgp-smime$ gpg --armor --export bob-lab@example.test > ~/cns-pgp-smime/bob_public.asc
anush@anusha:~/cns-pgp-smime$ ls -l ~/cns-pgp-smime/*.asc
-rw-r--r-- 1 anush anush 2464 Oct 31 16:35 /home/anush/cns-pgp-smime/alice_public.asc
-rw-r--r-- 1 anush anush 2460 Oct 31 16:35 /home/anush/cns-pgp-smime/bob_public.asc
anush@anusha:~/cns-pgp-smime$ head -n 4 ~/cns-pgp-smime/alice_public.asc
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQGNBGrmGNYBDACzdeVG4mW/Iu7Ig+im458a1vIs8ClmuQecxbAS/yEyL8vHkklO
VvR+PZ377wyOZPu1jjuG+FbWfoh17dgqPKiGKF2Yys7sMQkBePzTq1GzflaxL6m2
anush@anusha:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg
anush@anusha:~/cns-pgp-smime$ gpg --import ~/cns-pgp-smime/bob_public.asc
gpg: key 04376E3027539C54: public key "Bob CNS Lab <bob-lab@example.test>" imported
gpg: Total number processed: 1
gpg:               imported: 1
anush@anusha:~/cns-pgp-smime$ gpg --fingerprint bob-lab@example.test
pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      61B4 DEFA 64CB 87B4 79DE  9CB7 0437 6E30 2753 9C54
uid           [ unknown] Bob CNS Lab <bob-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
anush@anusha:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg
anush@anusha:~/cns-pgp-smime$ gpg --import ~/cns-pgp-smime/alice_public.asc
gpg: key 41C1BACD4CEEF430: public key "Alice CNS Lab <alice-lab@example.test>" imported
gpg: Total number processed: 1
gpg:               imported: 1
anush@anusha:~/cns-pgp-smime$ gpg --fingerprint alice-lab@example.test
pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      2EB7 F49C 23F3 2EA0 4E65  C5B8 41C1 BACD 4CEE F430
uid           [ unknown] Alice CNS Lab <alice-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
anush@anusha:~/cns-pgp-smime$
```

Figure 9: Step B5: message created and signed by Alice (detached ASCII-armored signature)

```
anush@anusha:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime$ printf 'CNS Lab: PGP secure e-mail test. Message ID: CNS-PGP-01.\n' > ~/cns-pgp-smime/
message.txt
anush@anusha:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg
anush@anusha:~/cns-pgp-smime$ gpg --armor --local-user alice-lab@example.test --detach-sign --output ~/cns-pgp-smime
/message.txt.asc ~/cns-pgp-smime/message.txt
Enter passphrase:
anush@anusha:~/cns-pgp-smime$ cp ~/cns-pgp-smime/message.txt* ~/cns-pgp-smime/transfer/
anush@anusha:~/cns-pgp-smime$ cat ~/cns-pgp-smime/transfer/message.txt ~/cns-pgp-smime/transfer/message.txt.asc
CNS Lab: PGP secure e-mail test. Message ID: CNS-PGP-01.
-----BEGIN PGP SIGNATURE-----

iQHLBAABCgA1FiEELrf0nCPzLqBOZcW4QcG6zUzu9DAFAmrmGN0XHGFsaWNlLWxh
YkBleGFtcGxlLnRlc3QACgkQQcG6zUzu9DCLLAv/UGuwuRz7in98HrywxpzTNfUz
ZQG17syHs80qbcwKx9iQtLxtsnAJ6a4kj17Vo4GoMbK6qUXxPEmPgzlBPOYhKaG1
RzyKq9p4ckq95cKyfpDGdE2HkLsldpRHNWYTCs+MDjea33yjzWhcUugtIG+z0N2I
xRBBV0PnNU6mdzIRBGmvWE/CkZoFlCChR17TxhRXUN82mgzj28ZyROBcETFWLlr+
LXCr8zdSi2frTdi4y+p76gTSdJP+ixYNxbnC5C7Uk8/5q4gWcvZMlv22P2wigxpd
PtSxA0MlGjYoQW5M7QMTcuZ30t4pHfuxR0KM2J5iKrRUwwoPcp3m3+r3KcmXPyHA
WQKdMRPQyEwg0bca8xIBrh/uWvlIzI9LkRAMMPFFDDjey2kOqsn1uGH/TrqkQiOH
BpMYPP1b/hHNcXBirH38e4zvGXfp7BckSYs48FCPXQvTr993s0O6nUf2m9RoyXRW
VKBDOFk7/KVb6IWWequ3XVuN48shcz3BpKOJU3NQ
=djEY
-----END PGP SIGNATURE-----
anush@anusha:~/cns-pgp-smime$
```

Figure 10: Step B6: Bob verifies Alice's detached signature - Good signature (trust warning noted separately)

```
anush@anusha:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg
anush@anusha:~/cns-pgp-smime$ gpg --verify ~/cns-pgp-smime/transfer/message.txt.asc ~/cns-pgp-smime/transfer/message
.txt
gpg: Signature made Sat Oct 31 16:35:41 2026 UTC
gpg:                using RSA key 2EB7F49C23F32EA04E65C5B841C1BACD4CEEF430
gpg:                issuer "alice-lab@example.test"
gpg: Good signature from "Alice CNS Lab <alice-lab@example.test>" [unknown]
gpg: WARNING: The key's User ID is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: 2EB7 F49C 23F3 2EA0 4E65  C5B8 41C1 BACD 4CEE F430
anush@anusha:~/cns-pgp-smime$ |
```

Figure 11: Step B7: Alice encrypts for Bob; Bob decrypts with his private key

```
anush@anusha:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg
anush@anusha:~/cns-pgp-smime$ gpg --armor --encrypt --recipient bob-lab@example.test --output ~/cns-pgp-smime/transf
er/message.txt.pgp ~/cns-pgp-smime/message.txt
gpg: 06368238FB4528ED: There is no assurance this key belongs to the named user

sub  rsa3072/06368238FB4528ED 2026-10-31 Bob CNS Lab <bob-lab@example.test>
 Primary key fingerprint: 61B4 DEFA 64CB 87B4 79DE  9CB7 0437 6E30 2753 9C54
      Subkey fingerprint: BCAE ABCE F922 7906 AAEE  DCA5 0636 8238 FB45 28ED

It is NOT certain that the key belongs to the person named
in the user ID.  If you *really* know what you are doing,
you may answer the next question with yes.

Use this key anyway? (y/N) y
anush@anusha:~/cns-pgp-smime$ head -n 6 ~/cns-pgp-smime/transfer/message.txt.pgp
-----BEGIN PGP MESSAGE-----

hQGMAwY2gjj7RSjtAQwAm82t5GX2MtyuPq9wVTDRKuVZU5HnwbhAfGfgv/p9dDox
uL+by5RmrkzrxBRQIIMY7pKILPypzgtxve9R+Vg8mxeXgzixmyFFgOr2ACn6Xhmd
3FZqXYbPd13Y3gxbkZGgkFoD3zk4c7nPpHMicTtF3RSHglEkREgZNFj6kIjszDCZ
5a1GtoIUDojkTLf2VZ8syVMFj1QfqYR6xbqQol56nyJJHMDBiGCKjBb7nKAWCSEf
anush@anusha:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg
anush@anusha:~/cns-pgp-smime$ gpg --output ~/cns-pgp-smime/decrypted_message.txt --decrypt ~/cns-pgp-smime/transfer/
message.txt.pgp
gpg: encrypted with rsa3072 key, ID 06368238FB4528ED, created 2026-10-31
      "Bob CNS Lab <bob-lab@example.test>"
Enter passphrase:
anush@anusha:~/cns-pgp-smime$ cat ~/cns-pgp-smime/decrypted_message.txt
CNS Lab: PGP secure e-mail test. Message ID: CNS-PGP-01.
anush@anusha:~/cns-pgp-smime$ |
```

Figure 12: Step B7: signed-and-encrypted message; Bob decrypts and sees the signature status

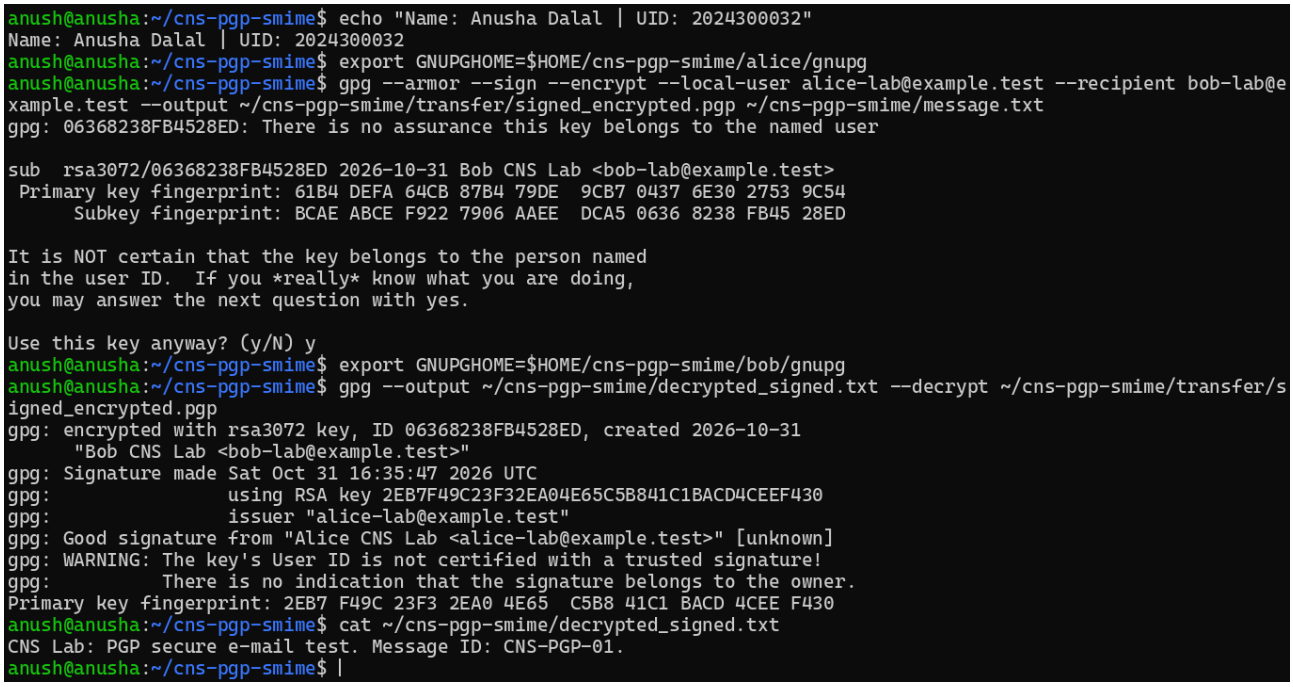
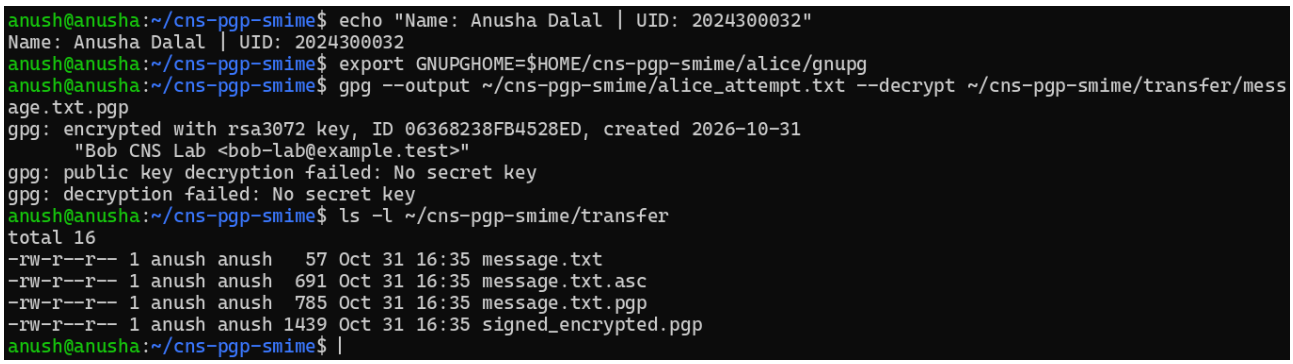


Figure 13: Step B8: tampered message checked against the original signature - BAD signature



Figure 14: Table 2 (optional row): decryption with a non-matching private key fails; transfer folder contents



4. S/MIME evidence (Part C)

A local lab CA issues X.509 certificates for Alice and Bob; OpenSSL's smime command signs, verifies, encrypts and decrypts the message.

Figure 15: Step C1: OpenSSL version and S/MIME workspace

```
anush@anusha:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
anush@anusha:~/cns-pgp-smime$ mkdir -p ~/cns-pgp-smime/smime
anush@anusha:~/cns-pgp-smime$ cd ~/cns-pgp-smime/smime
anush@anusha:~/cns-pgp-smime$ |
```

Figure 16: Step C2: lab CA certificate (CA:TRUE, keyCertSign, cRLSign)

```
anush@anusha:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime/smime$ openssl req -x509 -newkey rsa:3072 -keyout ca.key -out ca.crt -sha256 -days 365
-nodes -subj '/CN=CNS Lab CA' -addext 'basicConstraints=critical,CA:TRUE' -addext 'keyUsage=critical,keyCertSign,cRL
Sign'
.+.+.........+.....+.+.....+.+...+......+.....+......+..........+......+...+..+...+.+........+............+......+..
.+......+.+.........+...........+...+.......+..+...+....+...+...+...+......+.....+....+..+...+....+...+.............
.+......+..........+..+.+..+.............+.....+.+...+........+....+.....+.........+.......+..+......+....+...+.....
...+....+...........+....+..+.+.........+.....+......+.......+.....+......+.+.....+....+.....+.+.....+..........++++
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.+..+...........+......+.+...+...+.............+.+..
+...+...+.......+.....+...+....+...+...+.................+....+...+..+...+......+....+.....+.......................
......+...+....+.....+.+.....+...+......+................++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++*.....++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
....+.+......+.........++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.+..+.......+..............
....+...+.....+.+.....+......+....+...+.....+..........+...+...++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++*..+.......+........+..............+.......+......+.....+....+...+...+.....+......+....+..............+
.+..............+...+.........+.............+.+.......................+...+.....+.+..+...+..........+.....+.+....
..+........+....+.....+............+..............+.+..+..........+...........+.+.........+..+....+......+.........
........+...+...+...........+...+.+.........+.........+...+.........+...+.....+.....................+.+.....+...+..
....+..............+......+.........+.+...+......+.....+....+..+......+.......+.....+.......+......+.....+.......+.
.+...+...........+......+.......+.....+...........+.................+.......+...........+....+........+.+..+....+
.........+...........+....+......+........+......+.+...+......+..+.........+......+..................+......+.....+
.......+...........+..........+...........+.+............+...........+....+.........................+.........+...+
....+...+.....+.......+.....................+....+...........+.........+...+.............................+....+...
........+.+...+.............+...........+.+.....+...+....+...+.................+......+...+..+...+.......+...+...
........+.........+......+.......+...+...........+...+......+.+.....+......+.......+..............+.....+.........
...........+.+...+..+.................+..+...........+.......+...........+...+.........+.+..+.........+.........+
.......+..++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
-----
anush@anusha:~/cns-pgp-smime/smime$ openssl x509 -in ca.crt -noout -subject -issuer -dates -ext basicConstraints,key
Usage
subject=CN = CNS Lab CA
issuer=CN = CNS Lab CA
notBefore=Oct 31 16:35:48 2026 GMT
notAfter=Oct 31 16:35:48 2027 GMT
X509v3 Basic Constraints: critical
    CA:TRUE
X509v3 Key Usage: critical
    Certificate Sign, CRL Sign
anush@anusha:~/cns-pgp-smime/smime$ |
```

Figure 17: Step C3: Alice's key, CSR and certificate signed by the lab CA



Figure 18: Step C4: Bob's key, CSR and certificate; subject, issuer and validity of both certificates

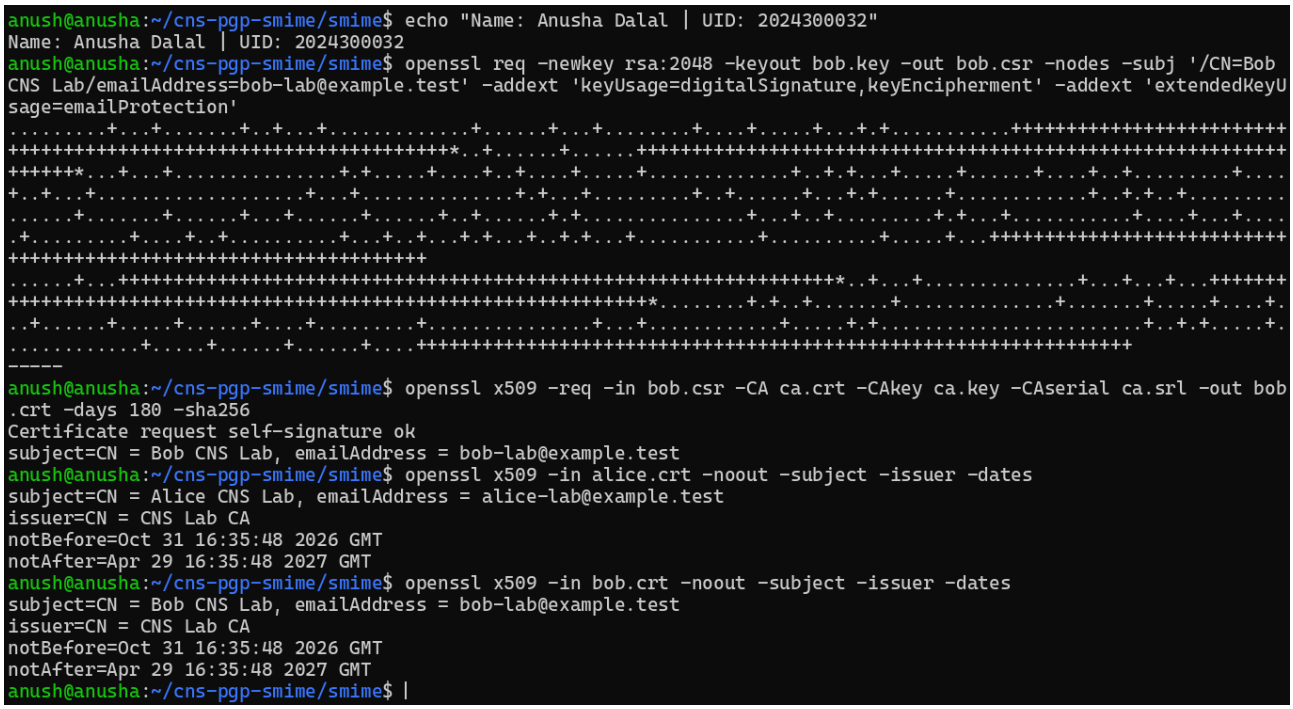


Figure 19: Certificate details: openssl x509 -text of Alice's certificate

```
anush@anusha:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime/smime$ openssl x509 -in alice.crt -noout -text
Certificate:
    Data:
        Version: 1 (0x0)
        Serial Number:
            24:1b:7c:d3:e8:31:eb:33:77:6a:f5:4f:97:8d:77:b0:4e:aa:e4:f7
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN = CNS Lab CA
        Validity
            Not Before: Oct 31 16:35:48 2026 GMT
            Not After : Apr 29 16:35:48 2027 GMT
        Subject: CN = Alice CNS Lab, emailAddress = alice-lab@example.test
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:a6:ac:74:4b:9e:d5:69:32:b2:c6:1e:f2:03:a9:
                    94:b9:f6:10:7e:a8:01:7b:c4:55:c8:d0:0d:ce:6c:
                    2e:e4:e7:a1:2a:d9:0a:bc:f4:e4:ba:50:58:ef:0d:
                    a3:77:af:99:01:4f:b7:c4:f2:a2:6b:f7:b8:ac:8b:
                    c4:a2:a8:73:1c:51:9b:24:42:a5:b6:4e:ff:6a:06:
                    f3:4e:63:67:72:04:8e:fc:f7:1c:82:54:96:af:a6:
                    9b:e1:d1:9d:0a:8b:b9:34:d6:83:98:35:ab:eb:b1:
                    c2:ce:32:f6:b7:3c:9b:33:13:0a:fd:49:74:3d:ca:
                    34:ba:04:10:eb:79:d1:b2:c7:4d:44:5f:f2:dd:9c:
                    c8:41:48:9f:41:50:20:1e:59:f2:a9:35:42:3d:9e:
                    4d:8d:4a:2c:2b:58:51:e0:51:af:f8:cd:ea:5a:e5:
                    59:6d:71:cd:c9:a1:9b:99:cc:c5:d4:29:c9:99:eb:
                    6a:03:f3:f8:e9:01:18:e6:79:36:72:3b:ad:93:cb:
                    ff:d8:b5:24:f4:c3:b8:c8:a9:46:52:0a:b7:9f:96:
                    78:ad:41:c5:ec:f2:cf:5e:51:7e:43:0e:ab:a0:1e:
                    8b:15:28:1f:d8:13:98:81:83:f3:20:37:f9:d2:60:
                    59:62:a8:b9:74:11:ca:95:38:a8:82:5d:6c:96:70:
                    47:61
                Exponent: 65537 (0x10001)
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        47:be:0b:1a:f9:86:48:ad:22:5f:aa:fe:dd:b2:20:cb:8a:52:
        bb:3f:4e:6e:65:ff:9f:92:f8:7f:80:65:95:34:02:e8:a5:32:
        91:93:e7:28:8b:2c:82:40:47:01:fa:c7:44:b6:81:35:14:65:
        52:5f:50:ba:cb:18:0e:30:31:54:34:33:98:e9:1b:2d:f2:9f:
        28:e9:ed:c9:85:3c:b7:69:da:d5:f4:eb:14:7c:93:ab:1a:73:
        01:b2:41:11:4f:be:16:f5:dc:2f:e9:14:b2:a3:99:81:e6:fb:
        58:97:48:cf:df:b6:8a:0d:e6:86:79:59:13:3e:7b:37:c5:32:
        f5:db:52:1f:11:7d:4a:47:a1:10:80:cc:51:85:8e:65:d7:81:
        00:3c:82:b9:cb:0c:5b:2e:bf:bc:09:51:eb:d5:25:85:46:9b:
        8d:ff:eb:06:69:93:59:30:9f:ab:c0:b3:b3:f1:24:bc:cb:ee:
        16:d3:51:9e:61:96:7e:33:a7:cb:96:1e:24:7d:66:5c:79:36:
        9d:65:b8:a8:35:14:89:69:0a:12:9b:d1:aa:c2:66:61:96:a6:
        2b:66:77:37:a6:e6:7f:5a:93:02:0c:4d:af:b0:9a:5c:0d:21:
        09:c0:f5:30:4a:7b:ea:0f:91:de:4e:f5:83:a8:2f:73:26:b3:
        6b:83:43:74:22:b1:28:85:b2:74:d3:15:01:00:ff:b1:0f:79:
        f4:c7:7c:4c:8f:ec:44:e8:c0:4d:48:0a:8d:d7:41:87:df:14:
        8f:60:15:43:07:08:83:6f:37:a7:57:7b:bb:52:fc:2e:21:c2:
        03:4a:b0:8f:94:96:06:b9:77:8a:ef:0b:1a:1d:0d:2c:f5:6a:
        20:60:f1:56:67:f8:e3:ca:d4:1c:d1:31:ca:33:ad:b1:4c:dc:
        fb:98:db:d8:7e:31:b8:b8:bb:86:d8:4a:c5:48:ca:19:c0:a5:
        44:6f:de:d6:e0:fa:21:66:6d:70:7a:8a:b8:44:a5:a9:da:3a:
        1d:ad:0f:f5:58:38
anush@anusha:~/cns-pgp-smime/smime$
```

Figure 20: Step C5: S/MIME signed message (first 15 lines)

```
anush@anusha:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime/smime$ printf 'CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.\n' > messa
ge.txt
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -sign -in message.txt -text -signer alice.crt -inkey alice.key -ou
t signed.eml
anush@anusha:~/cns-pgp-smime/smime$ head -n 15 signed.eml
MIME-Version: 1.0
Content-Type: multipart/signed; protocol="application/x-pkcs7-signature"; micalg="sha-256"; boundary="----19F2892412
1BE3D7436CAE9CBE358241"

This is an S/MIME signed message

------19F28924121BE3D7436CAE9CBE358241
Content-Type: text/plain

CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.

------19F28924121BE3D7436CAE9CBE358241
Content-Type: application/x-pkcs7-signature; name="smime.p7s"
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="smime.p7s"
anush@anusha:~/cns-pgp-smime/smime$
```

Figure 21: Step C6: signed message verified with the lab CA - Verification successful

```
anush@anusha:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -verify -in signed.eml -CAfile ca.crt -out verified_message.txt
Verification successful
anush@anusha:~/cns-pgp-smime/smime$ cat verified_message.txt
Content-Type: text/plain

CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.
anush@anusha:~/cns-pgp-smime/smime$
```

Figure 22: Steps C7-C8: encrypted for Bob (enveloped data) and decrypted with bob.key

```
anush@anusha:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -encrypt -in message.txt -text -out encrypted.eml bob.crt
anush@anusha:~/cns-pgp-smime/smime$ head -n 12 encrypted.eml
MIME-Version: 1.0
Content-Disposition: attachment; filename="smime.p7m"
Content-Type: application/x-pkcs7-mime; smime-type=enveloped-data; name="smime.p7m"
Content-Transfer-Encoding: base64

MIIB6QYJKoZIhvcNAQcDoIIB2jCCAdYCAQAxggFJMIIBRQIBADAtMBUxEzARBgNV
BAMMCkNOUyBMYWIgQ0ECFCQbfNPoMeszd2r1T5eNd7BOquT4MA0GCSqGSIb3DQEB
AQUABIIBAHIi/m6Y4JOKeiSD2Ria5YcYNKa0XqmN6auJt2OGYEna0UqqRVtGDnKf
QI+BYN9b6UExOq/tem6dPahgzDaX06uab0G6GVMUQCF93dvsTh4BSQ+FrupNCxBp
5J6U0lTBd8OXahyixXdy/vgo+/LIOzsHzLFrjlNYcuQBDpqzzXNnWXU8WiOnV4WW
QayW6rUY8FbwfvAJsgkjoYJzRXTD8e9JJTA/EUgyWlCBJDzp8GYzib3zUCseC1fG
bhFJs843opzCfyV1Utygrbp8LZsCKzXF6+PeQ7008Mw+5t3eD0PIbsDKC/VsbDmB
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -decrypt -in encrypted.eml -recip bob.crt -inkey bob.key -out smim
e_decrypted.txt
anush@anusha:~/cns-pgp-smime/smime$ cat smime_decrypted.txt
Content-Type: text/plain

CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.
anush@anusha:~/cns-pgp-smime/smime$ diff -u message.txt smime_decrypted.txt
--- message.txt 2026-10-31 16:35:48.951867314 +0000
+++ smime_decrypted.txt 2026-10-31 16:35:49.010015335 +0000
@@ -1 +1,3 @@
-CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.
+Content-Type: text/plain
+
+CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.
anush@anusha:~/cns-pgp-smime/smime$ tail -n +3 smime_decrypted.txt | tr -d '\r' | diff - message.txt && echo 'messag
e body is identical to message.txt'
message body is identical to message.txt
anush@anusha:~/cns-pgp-smime/smime$
```

Figure 23: Step C9: sign then encrypt; Bob decrypts and verifies the recovered signed message

```
anush@anusha:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -sign -in message.txt -text -signer alice.crt -inkey alice.key -ou
t signed_for_encryption.eml
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -encrypt -in signed_for_encryption.eml -out signed_encrypted.eml b
ob.crt
anush@anusha:~/cns-pgp-smime/smime$ head -n 8 signed_encrypted.eml
MIME-Version: 1.0
Content-Disposition: attachment; filename="smime.p7m"
Content-Type: application/x-pkcs7-mime; smime-type=enveloped-data; name="smime.p7m"
Content-Transfer-Encoding: base64

MIIL5AYJKoZIhvcNAQcDoIIL1TCCC9ECAQAxggFJMIIBRQIBADAtMBUxEzARBgNV
BAMMCkNOUyBMYWIgQ0ECFCQbfNPoMeszd2r1T5eNd7BOquT4MA0GCSqGSIb3DQEB
AQUABIIBADJBYfAoCzWutMMwkfWM4s7gLcyOhAPOn0f1mY0HE3STmwoNvK3FOUZu
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -decrypt -in signed_encrypted.eml -recip bob.crt -inkey bob.key -o
ut recovered_signed.eml
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -verify -in recovered_signed.eml -CAfile ca.crt -out recovered_ori
ginal.txt
Verification successful
anush@anusha:~/cns-pgp-smime/smime$ tail -n +3 recovered_original.txt | tr -d '\r' | diff - message.txt && echo 'mes
sage body is identical to message.txt'
message body is identical to message.txt
anush@anusha:~/cns-pgp-smime/smime$ |
```

Figure 24: Step C10: one character of the signed body changed - Verification failure (digest failure)

```
anush@anusha:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime/smime$ cp signed.eml signed_tampered.eml
anush@anusha:~/cns-pgp-smime/smime$ sed -i 's/S\/MIME secure e-mail test/S\/MIME secure e-mail tost/' signed_tampere
d.eml
anush@anusha:~/cns-pgp-smime/smime$ diff signed.eml signed_tampered.eml
9c9
< CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.
---
> CNS Lab: S/MIME secure e-mail tost. Message ID: CNS-SMIME-01.
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -verify -in signed_tampered.eml -CAfile ca.crt -out tampered_outpu
t.txt
Verification failure
4017640876780000:error:10800065:PKCS7 routines:PKCS7_signatureVerify:digest failure:../crypto/pkcs7/pk7_doit.c:1096:
4017640876780000:error:10800069:PKCS7 routines:PKCS7_verify:signature failure:../crypto/pkcs7/pk7_smime.c:345:
anush@anusha:~/cns-pgp-smime/smime$ |
```

Figure 25: Step C10: wrong CA / no CA file - certificate trust failure; with -noverify the signature itself is valid (1/2)

```
anush@anusha:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
anush@anusha:~/cns-pgp-smime/smime$ openssl req -x509 -newkey rsa:2048 -keyout other_ca.key -out other_ca.crt -nodes
 -days 30 -subj '/CN=Some Other CA'
.+........+.+......+...+..+.........+.+.........+........+.+......+........+...+...+....+..+...+......+.+.....+.+...
.....+.+..........++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.+.........+..+.+..+++++++++++
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.......+.......+.....+.+.........+.............+..........
..+......+...+............+..+.......+...+..+....+.........+........+.......+........+.......+.....+.............+..
+.+..........+.+...+..+...+.+.....+.+.....+.......+..+.........+....+.............................+.....+.+..+....
.....+....+......+........+...+....+.....+......+....+......+........+.......+...+.............................+...+..
.........+.+..............+...+.........................+......+..+...+..........+..................+...+.......+....
..........+..+.............+...+..................+.........+......+..+..........+...+..+....+.............
.+.........++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
.....+.+.................+..........+.....+.+.........+...+..+......++++++++++++++++++++++++++++++++++++++++++++++++
+++++++++++++++*......+....+..+.+........+......+..............++++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++*.........+..........+..............+..+......+.+...+.+.............+.+...........+...+...+....
...........+..+...+.......+.........+...+...+..............+..+.+......+......+........+......+....+.....+....+....
.+...........+...+.................+....+......+...+......+.....+...+...+.....+.....+.....+...+.....+.....+.....
.....................+.+......+......+..+...+...+....+...........+.......+........+......+...+...........+....+....
.+......+.+...+..........+........+....+......+........+..+.+........+.+..............+...........+...+.........
.............+...+..+.........+...+.+............+..+.+...........+....+.....+......+...+...........+.+..+...+....+
...................+.+..+.+..+.........+............+...........+.......+...+...........+...+.....+.......+..+...
.........+.............+..+.+..+.+......+..............+......+........+.+........+.......+......+..+..........+...
+.....+.......+........+.........+.+..+.+......+..............+.+...+...+.....+...+.......+.........+......+..+.....
.+...................+.......+.............+........+...+.........+.............+..+......+......+.....+.......
..+....+...+.....+.......+..+.+...+.....+.......+.........+........+.+......+......+.............................+..
.+.+..+....+......+..+......+....+..........+....+........+...................+......+...+..+...+.+........+.+.....
+.........+......+......+.++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
-----
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -verify -in signed.eml -CAfile other_ca.crt -out untrusted_output.
txt
Verification failure
4007334F0B770000:error:10800075:PKCS7 routines:PKCS7_verify:certificate verify error:../crypto/pkcs7/pk7_smime.c:296
:Verify error: unable to get local issuer certificate
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -verify -in signed.eml -out no_cafile_output.txt
Verification failure
40A7DA00F77D0000:error:10800075:PKCS7 routines:PKCS7_verify:certificate verify error:../crypto/pkcs7/pk7_smime.c:296
:Verify error: unable to get local issuer certificate
anush@anusha:~/cns-pgp-smime/smime$ openssl smime -verify -in signed.eml -noverify -out noverify_output.txt
Verification successful
anush@anusha:~/cns-pgp-smime/smime$ ls -l ~/cns-pgp-smime/smime
total 96
-rw-r--r-- 1 anush anush 1224 Oct 31 16:35 alice.crt
-rw-r--r-- 1 anush anush 1017 Oct 31 16:35 alice.csr
-rw------- 1 anush anush 1704 Oct 31 16:35 alice.key
-rw-r--r-- 1 anush anush 1220 Oct 31 16:35 bob.crt
-rw-r--r-- 1 anush anush 1013 Oct 31 16:35 bob.csr
-rw------- 1 anush anush 1704 Oct 31 16:35 bob.key
-rw-r--r-- 1 anush anush 1484 Oct 31 16:35 ca.crt
-rw------- 1 anush anush 2484 Oct 31 16:35 ca.key
-rw-r--r-- 1 anush anush   41 Oct 31 16:35 ca.srl
-rw-r--r-- 1 anush anush  863 Oct 31 16:35 encrypted.eml
-rw-r--r-- 1 anush anush   62 Oct 31 16:35 message.txt
-rw-r--r-- 1 anush anush    0 Oct 31 16:35 no_cafile_output.txt
-rw-r--r-- 1 anush anush   91 Oct 31 16:35 noverify_output.txt
-rw-r--r-- 1 anush anush 1123 Oct 31 16:35 other_ca.crt
-rw------- 1 anush anush 1704 Oct 31 16:35 other_ca.key
-rw-r--r-- 1 anush anush   91 Oct 31 16:35 recovered_original.txt
-rw-r--r-- 1 anush anush 2640 Oct 31 16:35 recovered_signed.eml
-rw-r--r-- 1 anush anush 2593 Oct 31 16:35 signed.eml
-rw-r--r-- 1 anush anush 4320 Oct 31 16:35 signed_encrypted.eml
-rw-r--r-- 1 anush anush 2593 Oct 31 16:35 signed_for_encryption.eml
```

Figure 26: Step C10: wrong CA / no CA file - certificate trust failure; with -noverify the signature itself is valid (2/2)

```
-rw-r--r-- 1 anush anush 2593 Oct 31 16:35 signed_tampered.eml
-rw-r--r-- 1 anush anush   91 Oct 31 16:35 smime_decrypted.txt
-rw-r--r-- 1 anush anush   91 Oct 31 16:35 tampered_output.txt
-rw-r--r-- 1 anush anush    0 Oct 31 16:35 untrusted_output.txt
-rw-r--r-- 1 anush anush   91 Oct 31 16:35 verified_message.txt
anush@anusha:~/cns-pgp-smime/smime$
```

5. Results, comparative analysis, discussion and conclusion

Table 2: PGP Results

Test	Expected result	Observed result / evidence
Public key export/import	Keys imported; fingerprints can be inspected	Both keys exported as ASCII-armored .asc files and imported into the other keyring ("public key imported"); fingerprints shown with `gpg --fingerprint`: Alice 2EB7 F49C 23F3 2EA0 4E65 C5B8 41C1 BACD 4CEE F430, Bob 61B4 DEFA 64CB 87B4 79DE 9CB7 0437 6E30 2753 9C54
Detached signature on original message	Good signature (trust status noted separately)	**Good signature from "Alice CNS Lab <alice-lab@example.test>" [unknown]**; separately: WARNING, the key is not certified with a trusted signature
Encrypt for Bob and decrypt as Bob	Decryption succeeds; content matches original	Decrypted with Bob's rsa3072 subkey; `decrypted_message.txt` = "CNS Lab: PGP secure e-mail test. Message ID: CNS-PGP-01."
Signed-and-encrypted message	Bob decrypts and signature status is displayed	Bob decrypted it and GnuPG showed **Good signature** from Alice (with the same trust warning); content matches
Modified message checked against original signature	Signature verification fails	**BAD signature from "Alice CNS Lab"** (tampered.txt had "ALTERED" appended)
Attempt to decrypt with a non-matching private key (optional)	Decryption fails	Alice tried to decrypt the message for Bob: **"decryption failed: No secret key"**

Table 3: S/MIME Results

Test	Expected result	Observed result / evidence
Inspect Alice/Bob certificate	Subject, issuer and validity displayed	Subject CN=Alice CNS Lab / CN=Bob CNS Lab with their e-mail addresses; issuer CN=CNS Lab CA; valid 180 days (Oct 31 16:35:48 2026 GMT to Apr 29 16:35:48 2027 GMT)
Verify original signed message using lab CA	Verification succeeds	**Verification successful**; recovered text written to verified_message.txt
Encrypt for Bob and decrypt using bob.key	Decryption succeeds; content matches original	encrypted.eml is enveloped-data (smime.p7m); decryption succeeded; message body identical to message.txt (only the MIME header added by `-text` and CRLF line endings differ)
Sign and encrypt workflow	Bob recovers the signed message	Bob decrypted signed_encrypted.eml, **Verification successful** on the recovered signed message, body identical to message.txt
Modified signed content	Signature verification fails	**Verification failure**: `PKCS7_signatureVerify: digest failure` and `signature failure` (one character changed: "test" → "tost")
Verify with untrusted/missing CA	Certificate trust validation fails or warns	With another CA or no -CAfile: **Verification failure: unable to get local issuer certificate** (a trust failure). With `-noverify` the same message verifies, so the signature itself is valid

Comparative analysis

Parameter	PGP / OpenPGP	S/MIME
Key / identity representation	OpenPGP public key and fingerprint	X.509 certificate containing public key and identity fields
Trust model	Often decentralized; fingerprints and user certifications	Usually CA-issued certificate chain and validation
Typical message representation	OpenPGP packet / ASCII-armored data	MIME with CMS/PKCS #7 structures
Confidentiality mechanism	Hybrid encryption; session key protected for recipient	Hybrid encryption; content key protected for recipient
Signature verification requires	Correct sender public key and valid signature	Sender certificate/public key, valid signature and applicable trust checks
Common deployment	OpenPGP-compatible clients and workflows	Enterprise e-mail and certificate-based environments
Observed challenge	Public-key authenticity and trust decisions	Certificate issuance, chain validation and lifecycle management

PGP was easier to set up in the lab: each user generated a key pair and exchanged public keys directly, with no third party. Its extra trust step is manual: each user must compare the full key fingerprint over a separate trusted channel (and could certify the key), otherwise GnuPG warns that the key is not certified. S/MIME needed more setup, because a CA had to be created first and had to issue a certificate for each user. In return, the recipient only has to trust the CA certificate, after which every certificate it issued can be validated automatically, but the CA's issuance, chain validation, renewal and revocation must be managed.

Answers to the discussion questions

1. PGP vs S/MIME: key management and trust model.

PGP users create and exchange their own key pairs, and trust is decentralised: you decide who to trust by checking fingerprints or through signatures from people you trust (web of trust). S/MIME uses X.509 certificates issued by a Certificate Authority; trust is hierarchical, so you trust the CA and therefore every certificate in its chain.

2. Why hybrid encryption?

Symmetric encryption (e.g. AES) is fast and suits large message content, but needs a shared key. Public-key encryption solves key distribution but is slow and limited in size. So a random one-time session key encrypts the message, and only that small key is encrypted with the recipient's public key.

3. Signing and verifying keys.

The sender signs with **her own private key**; anyone verifies with the **sender's public key** (in S/MIME, the public key inside her certificate).

4. Encrypting for Bob and decrypting.

The message is encrypted with **Bob's public key** (his certificate in S/MIME); Bob decrypts with **his own private key**.

5. Why never send private keys?

Whoever has a private key can decrypt everything sent to that person and sign messages in their name. The transfer folder and e-mail are insecure channels, so a sent private key must be assumed compromised; confidentiality, authentication and non-repudiation would all be lost.

6. Valid signature vs trusted identity.

A valid signature only proves the message was signed by the private key matching the public key used, and was not changed. It does not prove whose key it is. Trust in the identity needs separate checking: fingerprint verification

(PGP) or a valid CA chain (S/MIME). That is why GnuPG said "Good signature" and also "not certified with a trusted signature".

7. Why import without fingerprint check enables MITM.

An attacker on the channel could replace Alice's public key with their own key carrying Alice's name. Bob would then accept the attacker's signatures as Alice's, and messages Bob encrypts "to Alice" could be read by the attacker. Comparing the full fingerprint over a trusted channel detects the substitution.

8. Role of the CA certificate in S/MIME verification.

It is the trust anchor. OpenSSL checks that Alice's certificate was signed by a CA in the -CAfile; with the CA it reported "Verification successful", and without it "unable to get local issuer certificate", even though the signature itself was valid.

9. What happened when the message was modified?

Verification failed: GnuPG reported "BAD signature" and OpenSSL "digest failure". The signature is over a hash of the exact content; any change gives a different hash that no longer matches the signed hash, so integrity violation is detected.

10. Why is a lab/self-signed certificate not trusted in production?

Mail clients trust only CAs in their built-in trust store, which are audited and verify identities. Our lab CA is not in that store, and anyone can create a self-signed CA claiming any name, so trusting it automatically would let attackers forge identities.

11. Can a signed message also be encrypted?

Yes. Sign first, then encrypt: the signature covers the real content and is itself hidden inside the encryption, so only Bob sees who signed and what was signed. Bob decrypts first, then verifies. We did this in both parts (`--sign --encrypt` in PGP; sign then encrypt in S/MIME).

12. One advantage and one limitation of each.

- **PGP:** advantage: free, no CA needed, users control their own keys. Limitation: key authenticity depends on users checking fingerprints; weak usability for non-experts.
- **S/MIME:** advantage: built into enterprise mail clients, and automatic validation through a CA chain. Limitation: requires obtaining, renewing and revoking certificates from a CA (cost and management overhead).

Conclusion (150–200 words)

Conclusion
[hand-written page attached]

6. Commands used and deviations

Commands used

Step	Commands

Table 1	`lsb_release -d 2>/dev/null || grep PRETTY_NAME /etc/os-release`
	`uname -r`
	`gpg --version | head -3`
	`openssl version`
	`lscpu | grep -E 'Model name|^CPU\(s\)'`
	`free -h | head -2`
	`date`

B1	`gpg --version`

B2	`mkdir -p ~/cns-pgp-smime/{alice,bob,transfer}`
	`cd ~/cns-pgp-smime`
	`mkdir -p ~/cns-pgp-smime/alice/gnupg ~/cns-pgp-smime/bob/gnupg`
	`chmod 700 ~/cns-pgp-smime/alice/gnupg ~/cns-pgp-smime/bob/gnupg`
	`echo "pinentry-mode loopback" > ~/cns-pgp-smime/alice/gnupg/gpg.conf`
	`echo "pinentry-mode loopback" > ~/cns-pgp-smime/bob/gnupg/gpg.conf`
	`ls -la ~/cns-pgp-smime ~/cns-pgp-smime/alice ~/cns-pgp-smime/bob`

B3 (Alice)	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --full-generate-key`
	`gpg --list-keys`
	`gpg --list-secret-keys`

B3 (Bob)	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --full-generate-key`
	`gpg --list-keys`

B4	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --armor --export alice-lab@example.test > ~/cns-pgp-smime/alice_public.asc`
	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --armor --export bob-lab@example.test > ~/cns-pgp-smime/bob_public.asc`
	`ls -l ~/cns-pgp-smime/*.asc`
	`head -n 4 ~/cns-pgp-smime/alice_public.asc`
	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --import ~/cns-pgp-smime/bob_public.asc`
	`gpg --fingerprint bob-lab@example.test`
	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --import ~/cns-pgp-smime/alice_public.asc`
	`gpg --fingerprint alice-lab@example.test`

B5	`printf 'CNS Lab: PGP secure e-mail test. Message ID: CNS-PGP-01.\n' >`
	`~/cns-pgp-smime/message.txt`
	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --armor --local-user alice-lab@example.test --detach-sign --output`
	`~/cns-pgp-smime/message.txt.asc ~/cns-pgp-smime/message.txt`
	`cp ~/cns-pgp-smime/message.txt* ~/cns-pgp-smime/transfer/`
	`cat ~/cns-pgp-smime/transfer/message.txt ~/cns-pgp-smime/transfer/message.txt.asc`

B6	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --verify ~/cns-pgp-smime/transfer/message.txt.asc`
	`~/cns-pgp-smime/transfer/message.txt`

B7	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --armor --encrypt --recipient bob-lab@example.test --output`
	`~/cns-pgp-smime/transfer/message.txt.pgp ~/cns-pgp-smime/message.txt`
	`head -n 6 ~/cns-pgp-smime/transfer/message.txt.pgp`
	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --output ~/cns-pgp-smime/decrypted_message.txt --decrypt`
	`~/cns-pgp-smime/transfer/message.txt.pgp`
	`cat ~/cns-pgp-smime/decrypted_message.txt`

B7 (sign + encrypt)	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --armor --sign --encrypt --local-user alice-lab@example.test --recipient bob-lab@example.test --output ~/cns-pgp-smime/transfer/signed_encrypted.pgp ~/cns-pgp-smime/message.txt`
	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --output ~/cns-pgp-smime/decrypted_signed.txt --decrypt`
	`~/cns-pgp-smime/transfer/signed_encrypted.pgp`
	`cat ~/cns-pgp-smime/decrypted_signed.txt`

B8	`cp ~/cns-pgp-smime/message.txt ~/cns-pgp-smime/tampered.txt`
	`printf 'ALTERED' >> ~/cns-pgp-smime/tampered.txt`
	`cat ~/cns-pgp-smime/tampered.txt; echo`
	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --verify ~/cns-pgp-smime/transfer/message.txt.asc ~/cns-pgp-smime/tampered.txt`

Table 2 (optional)	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --output ~/cns-pgp-smime/alice_attempt.txt --decrypt`
	`~/cns-pgp-smime/transfer/message.txt.pgp`
	`ls -l ~/cns-pgp-smime/transfer`

C1	`openssl version`
	`mkdir -p ~/cns-pgp-smime/smime`
	`cd ~/cns-pgp-smime/smime`

C2	`openssl req -x509 -newkey rsa:3072 -keyout ca.key -out ca.crt -sha256 -days 365 -nodes -subj '/CN=CNS Lab CA' -addext 'basicConstraints=critical,CA:TRUE' -addext 'keyUsage=critical,keyCertSign,cRLSign'` `openssl x509 -in ca.crt -noout -subject -issuer -dates -ext basicConstraints,keyUsage`
C3	`openssl req -newkey rsa:2048 -keyout alice.key -out alice.csr -nodes -subj '/CN=Alice CNS Lab/emailAddress=alice-lab@example.test' -addext 'keyUsage=digitalSignature,keyEncipherment' -addext 'extendedKeyUsage=emailProtection'` `openssl x509 -req -in alice.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out alice.crt -days 180 -sha256`

C4	`openssl req -newkey rsa:2048 -keyout bob.key -out bob.csr -nodes -subj '/CN=Bob CNS Lab/emailAddress=bob-lab@example.test' -addext 'keyUsage=digitalSignature,keyEncipherment' -addext 'extendedKeyUsage=emailProtection'`
	`openssl x509 -req -in bob.csr -CA ca.crt -CAkey ca.key -CAserial ca.srl -out bob.crt -days 180 -sha256`
	`openssl x509 -in alice.crt -noout -subject -issuer -dates`
	`openssl x509 -in bob.crt -noout -subject -issuer -dates`

C4 (details)	`openssl x509 -in alice.crt -noout -text`

C5	`printf 'CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.\n' > message.txt`
	`openssl smime -sign -in message.txt -text -signer alice.crt -inkey alice.key -out signed.eml`
	`head -n 15 signed.eml`

C6	`openssl smime -verify -in signed.eml -CAfile ca.crt -out verified_message.txt` `cat verified_message.txt`

C7–C8	`openssl smime -encrypt -in message.txt -text -out encrypted.eml bob.crt`
	`head -n 12 encrypted.eml`
	`openssl smime -decrypt -in encrypted.eml -recip bob.crt -inkey bob.key -out smime_decrypted.txt`
	`cat smime_decrypted.txt`
	`diff -u message.txt smime_decrypted.txt`
	`tail -n +3 smime_decrypted.txt | tr -d '\r' | diff - message.txt && echo 'message body is identical to message.txt'`

C9	`openssl smime -sign -in message.txt -text -signer alice.crt -inkey alice.key -out signed_for_encryption.eml`
	`openssl smime -encrypt -in signed_for_encryption.eml -out signed_encrypted.eml bob.crt`
	`head -n 8 signed_encrypted.eml`
	`openssl smime -decrypt -in signed_encrypted.eml -recip bob.crt -inkey bob.key -out recovered_signed.eml`
	`openssl smime -verify -in recovered_signed.eml -CAfile ca.crt -out recovered_original.txt`
	`tail -n +3 recovered_original.txt | tr -d '\r' | diff - message.txt && echo 'message body is identical to message.txt'`

C10 (tamper)	`cp signed.eml signed_tampered.eml`
	`sed -i 's/S\/MIME secure e-mail test/S\/MIME secure e-mail tost/' signed_tampered.eml`
	`diff signed.eml signed_tampered.eml`
	`openssl smime -verify -in signed_tampered.eml -CAfile ca.crt -out tampered_output.txt`

C10 (trust)	`openssl req -x509 -newkey rsa:2048 -keyout other_ca.key -out other_ca.crt -nodes -days 30 -subj '/CN=Some Other CA'`
	`openssl smime -verify -in signed.eml -CAfile other_ca.crt -out untrusted_output.txt`
	`openssl smime -verify -in signed.eml -out no_cafile_output.txt`
	`openssl smime -verify -in signed.eml -noverify -out noverify_output.txt`
	`ls -l ~/cns-pgp-smime/smime`

Deviations caused by software versions or the environment

- **Passphrase prompt in the terminal:** without a graphical desktop, GnuPG's pinentry pop-up is not available, so each keyring's gpg.conf contains `pinentry-mode loopback`; GnuPG then asks for the passphrase directly in the terminal (the passphrase is not echoed). The key-generation answers were exactly those in the manual (RSA and RSA, 3072 bits, expiry 1 year, the given names and e-mail addresses).
- **"Use this key anyway? (y/N)":** when Alice encrypts for Bob, GnuPG 2.4 asks this because Bob's imported key is not yet certified; it was answered y after the fingerprint was compared (Step B4).
- **C10 edit with sed instead of a text editor:** one character of the signed body was changed with `sed` ("test" to "tost"); `diff` shows that only that line changed and the MIME structure is intact.
- **Certificate extensions:** in OpenSSL 3.0, `openssl x509 -req` does not copy extensions from the CSR unless `-copy_extensions copy` is given, so alice.crt and bob.crt contain no keyUsage/extendedKeyUsage. S/MIME still works, because a certificate without these extensions is not restricted.

- **Decrypted file vs message.txt:** the `-text` option wraps the message in a `Content-Type: text/plain` MIME header and S/MIME uses CRLF line endings, so the files differ by the header and line endings only; the message body was compared with `tail -n +3 | tr -d '\r' | diff` and is identical.
- **Untrusted CA test:** in addition to omitting -CAfile, a second unrelated CA ("Some Other CA") was used, and `-noverify` was run to show that the failure is a certificate-trust failure, not a content failure.